

DEVSPHERE

INTERNSHIP PROGRAM

WEEK 2 TASK REPORT

Artificial Intelligence (AI) & Machine Learning (ML)

Intern	Saud Masood
Domain	Artificial Intelligence & Machine Learning
Internship	DevSphere Internship Program
Week	02
Report Date	13 September 2026

Submission Deadline: 14 September 2026

Submission Email: devsphere1.info@gmail.com

Prepared and submitted by Saud Masood

1. Artificial Intelligence (AI) — Week 2

Topic: Search Algorithms

Task: Implement search algorithms.

Requirements:

- Linear Search
- Binary Search
- Show working with examples
- Compare performance (basic)

Output: Python program with results.

Aim

In this task, I implemented two common search techniques in Python: Linear Search and Binary Search. I used a simple sorted list and searched for the value 70 so I could see how both methods work and compare their results and execution time.

Implementation Explanation

For the example, I used the sorted list [10, 20, 30, 40, 50, 60, 70, 80, 90, 100] and searched for 70. Linear Search starts from the beginning and checks each value until it finds the target. Binary Search works differently: because the list is sorted, it checks the middle value and keeps reducing the search range until the target is found.

Python Code

```
import time

print("=====")
print("  AI WEEK 2 - SEARCH ALGORITHMS")
print("=====")

def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

def binary_search(arr, target):
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
```

```

        left = mid + 1
    else:
        right = mid - 1

    return -1

data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

target = 70

print("\n===== EXAMPLE DATA =====")
print("Data:", data)
print("Target:", target)

print("\n===== LINEAR SEARCH =====")

start = time.perf_counter()
linear_result = linear_search(data, target)
linear_time = time.perf_counter() - start

if linear_result != -1:
    print("Target found at index:", linear_result)
else:
    print("Target not found")

print("Time taken:", linear_time, "seconds")

print("\n===== BINARY SEARCH =====")

start = time.perf_counter()
binary_result = binary_search(data, target)
binary_time = time.perf_counter() - start

if binary_result != -1:
    print("Target found at index:", binary_result)
else:
    print("Target not found")

print("Time taken:", binary_time, "seconds")

print("\n===== PERFORMANCE COMPARISON =====")

print("Linear Search:")

```

```
print("Time Complexity: O(n)")
print("Checks elements one by one.")

print("\nBinary Search:")
print("Time Complexity: O(log n)")
print("Divides the sorted list into smaller parts.")

print("\n===== RESULT =====")

if linear_result != -1 and binary_result != -1:
    print("Both algorithms successfully found the target.")

if binary_time < linear_time:
    print("Binary Search was faster in this example.")
else:
    print("Linear Search was faster in this example.")

print("\nAI Week 2 Search Algorithms completed successfully!")
```

Code Screen Shots:

```
32
33 target = 70
34
35 print("\n===== EXAMPLE DATA =====")
36 print("Data:", data)
37 print("Target:", target)
38
39 print("\n===== LINEAR SEARCH =====")
40
41 start = time.perf_counter()
42 linear_result = linear_search(data, target)
43 linear_time = time.perf_counter() - start
44
45 if linear_result != -1:
46 | | print("Target found at index:", linear_result)
47 else:
48 | | print("Target not found")
49
50 print("Time taken:", linear_time, "seconds")
51
52 print("\n===== BINARY SEARCH =====")
53
54 start = time.perf_counter()
55 binary_result = binary_search(data, target)
56 binary_time = time.perf_counter() - start
57
58 if binary_result != -1:
59 | | print("Target found at index:", binary_result)
60 else:
61 | | print("Target not found")
62
63 print("Time taken:", binary_time, "seconds")
64
65 print("\n===== PERFORMANCE COMPARISON =====")
```

```
64
65 print("\n===== PERFORMANCE COMPARISON =====")
66
67 print("Linear Search:")
68 print("Time Complexity: O(n)")
69 print("Checks elements one by one.")
70
71 print("\nBinary Search:")
72 print("Time Complexity: O(log n)")
73 print("Divides the sorted list into smaller parts.")
74
75 print("\n===== RESULT =====")
76
77 if linear_result != -1 and binary_result != -1:
78     print("Both algorithms successfully found the target.")
79
80 if binary_time < linear_time:
81     print("Binary Search was faster in this example.")
82 else:
83     print("Linear Search was faster in this example.")
84
85 print("\nAI Week 2 Search Algorithms completed successfully!")
```

2. AI Task — Actual Google Colab Output

```
✓ 0s 82 else:
      83 | | print("Linear Search was faster in this example.")
      84
      85 print("\nAI Week 2 Search Algorithms completed successfully!")

✓
=====
AI WEEK 2 - SEARCH ALGORITHMS
=====

===== EXAMPLE DATA =====
Data: [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
Target: 70

===== LINEAR SEARCH =====
Target found at index: 6
Time taken: 5.924799995682406e-05 seconds

===== BINARY SEARCH =====
Target found at index: 6
Time taken: 5.975600004148873e-05 seconds

===== PERFORMANCE COMPARISON =====
Linear Search:
Time Complexity: O(n)
Checks elements one by one.

Binary Search:
Time Complexity: O(log n)
Divides the sorted list into smaller parts.

===== RESULT =====
Both algorithms successfully found the target.
Linear Search was faster in this example.
```

Result Explanation

The result shows that both algorithms successfully found 70 at index 6. In this particular Google Colab run, Linear Search took 5.924799995682406e-05 seconds and Binary Search took 5.975600004148873e-05 seconds, so Linear Search was slightly faster for this small example. The important comparison is that Linear Search has $O(n)$ time complexity, while Binary Search has $O(\log n)$ time complexity and is designed for sorted data.

3. Machine Learning (ML) — Week 2

Topic: Data Visualization

Task: Visualize dataset.

Requirements:

- Use Matplotlib / Seaborn
- Create graphs (bar, line, pie)
- Show insights from data
- Clean and labeled charts

Output: Graphs with explanation.

Aim

In this task, I used Python and Matplotlib to turn the cleaned student data into simple visual charts. I created a bar chart, a line chart, and a pie chart so the results could be understood more easily instead of looking only at the raw numbers.

Dataset Used

I used the cleaned_students.csv file from my previous data-cleaning work. The program reads the Math, English, Science, and Gender columns, calculates the subject averages, and then uses those results to create the required visualizations.

Python Code

```
import pandas as pd

import matplotlib.pyplot as plt

print("=====")

print(" ML WEEK 2 - DATA VISUALIZATION")

print("=====")

df = pd.read_csv("cleaned_students.csv")

print("\nDataset loaded successfully!")

print("\n===== DATA =====")

print(df)

subject_means = df[["Math", "English", "Science"]].mean()

print("\n===== SUBJECT AVERAGES =====")

print(subject_means)
```



```
print("\n===== INSIGHTS =====")

highest_subject = subject_means.idxmax()

lowest_subject = subject_means.idxmin()

print(
    "Highest average score:",
    highest_subject,
    "=",
    round(subject_means[highest_subject], 2)
)

print(
    "Lowest average score:",
    lowest_subject,
    "=",
    round(subject_means[lowest_subject], 2)
)

print(
    "Overall average score:",
    round(subject_means.mean(), 2)
)

gender_counts = df["Gender"].value_counts()

print("\nGender distribution:")

print(gender_counts)

print("\n===== BAR CHART =====")

plt.figure(figsize=(8, 5))

plt.bar(
    subject_means.index,
    subject_means.values
)

plt.title("Average Score by Subject")
plt.xlabel("Subject")
plt.ylabel("Average Score")
```

```
plt.tight_layout()

plt.show()


print("\n===== LINE CHART =====")

plt.figure(figsize=(8, 5))

plt.plot(
    subject_means.index,
    subject_means.values,
    marker="o"
)

plt.title("Average Student Performance")
plt.xlabel("Subject")
plt.ylabel("Average Score")

plt.grid(True)

plt.tight_layout()

plt.show()


print("\n===== PIE CHART =====")

plt.figure(figsize=(7, 7))

plt.pie(
    gender_counts.values,
    labels=gender_counts.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Student Gender Distribution")

plt.tight_layout()
```

```
plt.show()
```

```
print("\n=====")
```

```
print(" DATA VISUALIZATION COMPLETED")
```

```
print("=====")
```

Code Screen Shots:



```
2] 3s 1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 print("=====")
5 print(" ML WEEK 2 - DATA VISUALIZATION")
6 print("=====")
7
8 df = pd.read_csv("cleaned_students.csv")
9
10 print("\nDataset loaded successfully!")
11
12 print("\n===== DATA =====")
13 print(df)
14
15 subject_means = df[["Math", "English", "Science"]].mean()
16
17 print("\n===== SUBJECT AVERAGES =====")
18 print(subject_means)
19
20 print("\n===== INSIGHTS =====")
21
22 highest_subject = subject_means.idxmax()
23 lowest_subject = subject_means.idxmin()
24
25 print(
26     "Highest average score:",
27     highest_subject,
28     "=",
29     round(subject_means[highest_subject], 2)
30 )
31
32 print(
33     "Lowest average score:",
34     lowest_subject,
35     "=",
```

```

28 |     ,
29 |     round(subject_means[highest_subject], 2)
30 | )
31 |
32 | print(
33 |     "Lowest average score:",
34 |     lowest_subject,
35 |     "=",
36 |     round(subject_means[lowest_subject], 2)
37 | )
38 |
39 | print(
40 |     "Overall average score:",
41 |     round(subject_means.mean(), 2)
42 | )
43 |
44 | gender_counts = df["Gender"].value_counts()
45 |
46 | print("\nGender distribution:")
47 | print(gender_counts)
48 |
49 | print("\n===== BAR CHART =====")
50 |
51 | plt.figure(figsize=(8, 5))
52 |
53 | plt.bar(
54 |     subject_means.index,
55 |     subject_means.values
56 | )
57 |
58 | plt.title("Average Score by Subject")
59 | plt.xlabel("Subject")
60 | plt.ylabel("Average Score")
61 |
62 | plt.tight_layout()
63 | plt.show()
64 |

```

```

60 plt.ylabel("Average Score")
61
62 plt.tight_layout()
63 plt.show()
64
65
66 print("\n===== LINE CHART =====")
67
68 plt.figure(figsize=(8, 5))
69
70 plt.plot(
71     subject_means.index,
72     subject_means.values,
73     marker="o"
74 )
75
76 plt.title("Average Student Performance")
77 plt.xlabel("Subject")
78 plt.ylabel("Average Score")
79
80 plt.grid(True)
81
82 plt.tight_layout()
83 plt.show()
84
85
86 print("\n===== PIE CHART =====")
87
88 plt.figure(figsize=(7, 7))
89
90 plt.pie(
91     gender_counts.values,
92     labels=gender_counts.index,
93     autopct="%1.1f%%",
94     startangle=90
95 )
--

```

```

92     labels=gender_counts.index,
93     autopct="%1.1f%",
94     startangle=90
95 )
96
97 plt.title("Student Gender Distribution")
98
99 plt.tight_layout()
100 plt.show()
101
102
103 print("\n=====")
104 print("  DATA VISUALIZATION COMPLETED")
105 print("=====")

```

4. ML Task — Actual Dataset Output

```
[2] 104 print("... DATA VISUALIZATION COMPLETED")
✓ 3s 105 print("=====")

...
=====
ML WEEK 2 - DATA VISUALIZATION
=====

Dataset loaded successfully!

===== DATA =====
      Name  Age  Gender  Math  English  Science
0     Ali   20   Male   85.00  78.000000     90
1   Ahmed   21   Male   72.00  80.000000     75
2     Sara   20  Female   95.00  88.000000     92
3   Ayesha   22  Female   81.75  91.000000     89
4    Usman   21   Male   68.00  82.666667     70
5    Hamza   20   Male   80.00  75.000000     85
6   Fatima   21  Female   90.00  85.000000     94
7    Bilal   22   Male   76.00  82.000000     78
8     Hina   20  Female   81.75  89.000000     91
9     Zain   21   Male   88.00  76.000000     84

===== SUBJECT AVERAGES =====
Math          81.750000
English       82.666667
Science       84.800000
dtype: float64

===== INSIGHTS =====
Highest average score: Science = 84.8
Lowest average score: Math = 81.75
Overall average score: 83.07
```

```
7    Bilal    22    Male    76.00    82.000000    78
8    Hina    20    Female    81.75    89.000000    91
9    Zain    21    Male    88.00    76.000000    84
```

```
===== SUBJECT AVERAGES =====
```

```
Math      81.750000
```

```
English   82.666667
```

```
Science   84.800000
```

```
dtype: float64
```

```
===== INSIGHTS =====
```

```
Highest average score: Science = 84.8
```

```
Lowest average score: Math = 81.75
```

```
Overall average score: 83.07
```

```
Gender distribution:
```

```
Gender
```

```
Male      6
```

```
Female    4
```

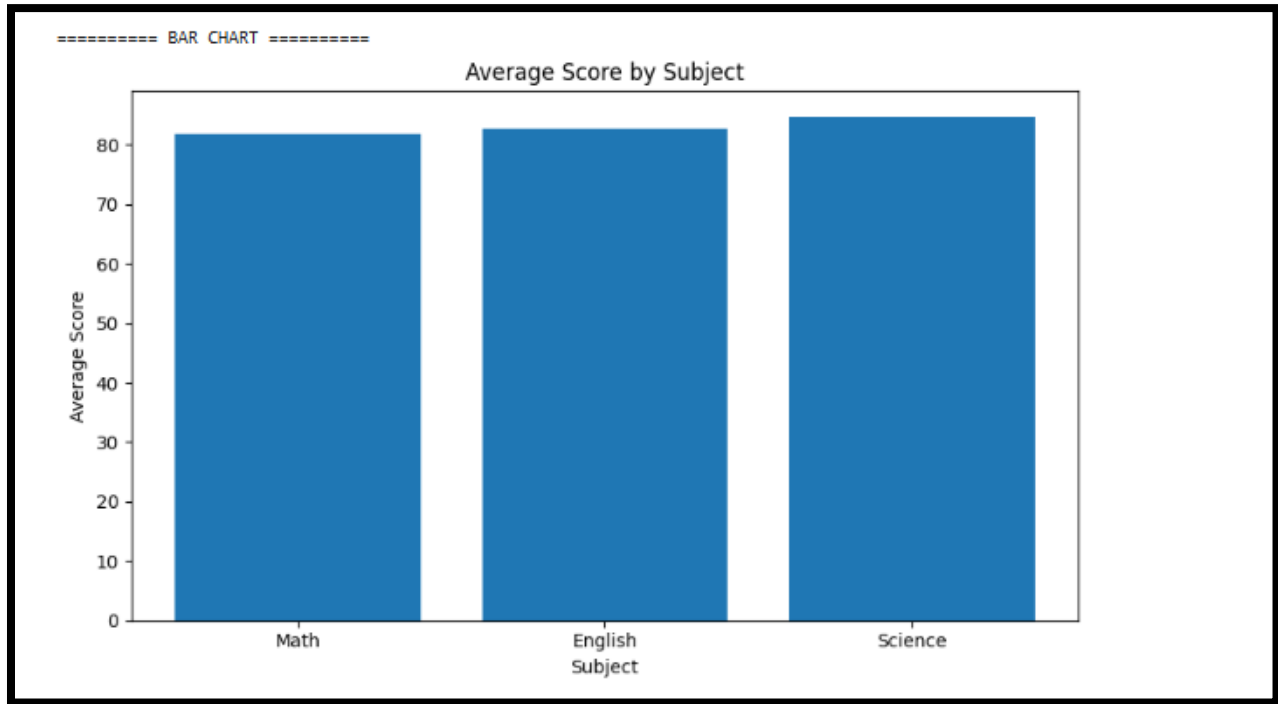
```
Name: count, dtype: int64
```

```
===== BAR CHART =====
```

5. ML Task — Bar and Line Charts

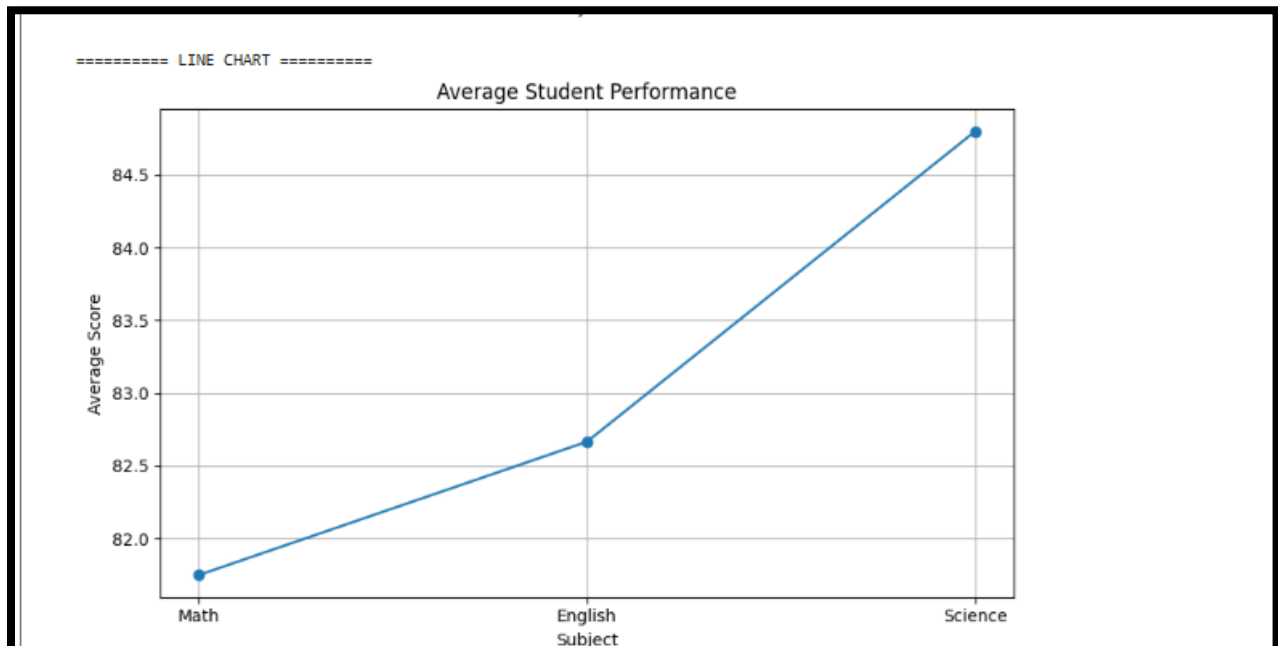
Bar Chart — Average Score by Subject

The bar chart makes the difference between the three subject averages easy to see. Science has the highest average score at 84.80, while Math has the lowest at 81.75.



Line Chart — Average Student Performance

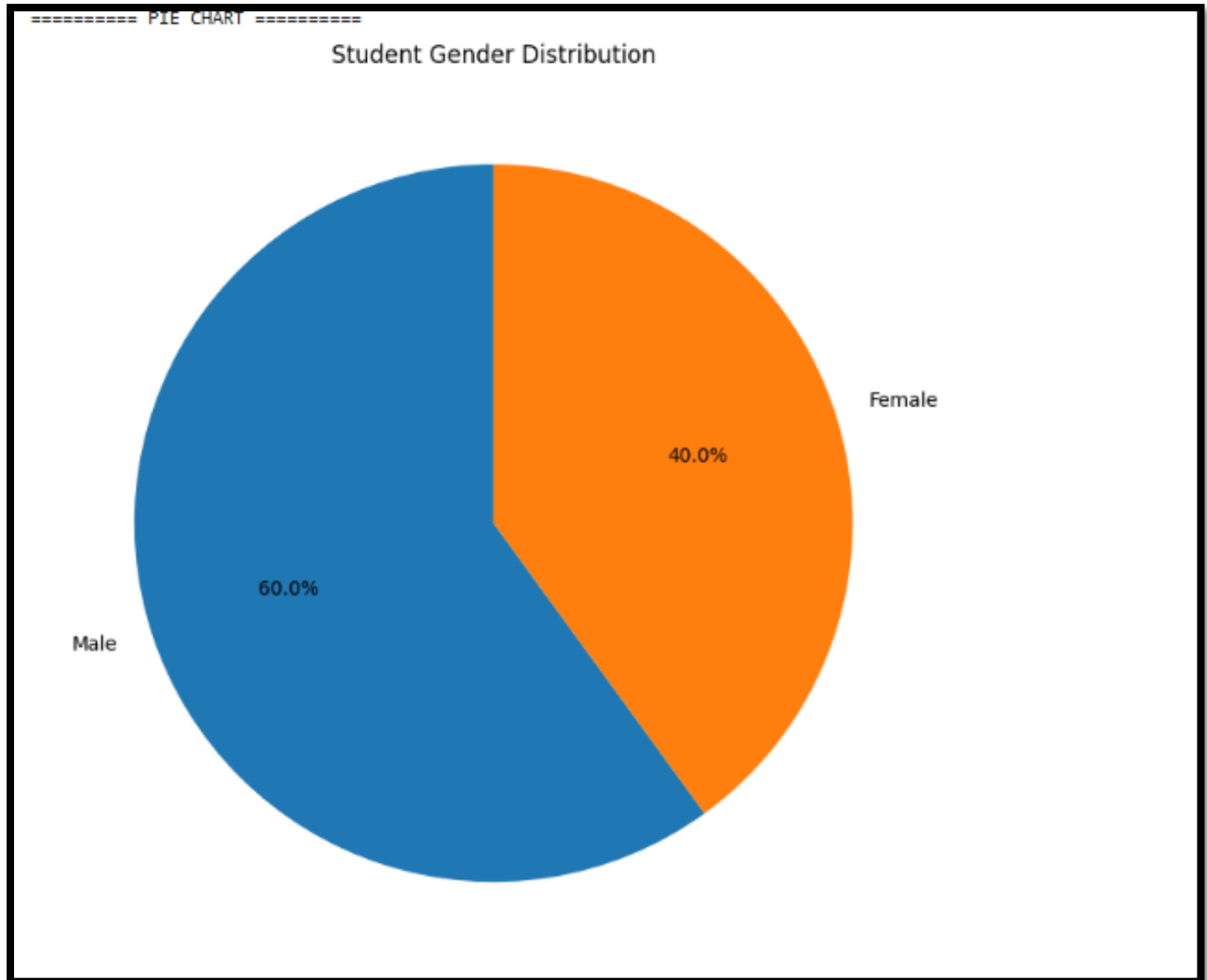
The line chart gives another view of the subject averages. It shows the scores increasing from Math to English and then to Science, making the overall pattern easy to notice.



6. ML Task — Pie Chart and Insights

Pie Chart — Student Gender Distribution

The pie chart shows the gender distribution in the dataset. There are 6 male students and 4 female students, so the chart represents the data as 60% male and 40% female.



Data Insights

- Highest average score: Science = 84.80
- Lowest average score: Math = 81.75
- Overall average score: 83.07
- Gender distribution: Male = 6, Female = 4

Completion Status

AI Week 2 — Search Algorithms: Completed

ML Week 2 — Data Visualization: Completed

Submission Deadline: 14 September 2026

Submission Email: devsphere1.info@gmail.com

DevSphere Internship Program • Week 02 • 5 September 2026

Submitted & Prepared by Saud Masood